

RAG System Implementation Plan

Goal: Build a RAG system over an AI governance document corpus that retrieves relevant chunks via hybrid retrieval (dense + sparse + MMR), fuses results with RRF, and generates grounded, source-cited answers using Gemini 2.5 Flash.

Architecture: Documents are loaded from `data/documents/`, chunked at 1000 chars with 200 overlap, embedded with `all-MiniLM-L6-v2`, and persisted in ChromaDB. At query time, three retrievers (dense similarity, BM25 sparse, ChromaDB MMR) each return top-20 candidates; RRF fuses the ranked lists into a single scored set; a custom MMR pass selects 8 diverse top chunks; Gemini 2.5 Flash generates a grounded answer constrained to retrieved context with source citations.

Tech Stack: Python 3.11+, LangChain, ChromaDB, sentence-transformers (`all-MiniLM-L6-v2`), rank-bm25, langchain-google-genai, scikit-learn, numpy, pytest, PyPDF

File Map

File	Responsibility
<code>requirements.txt</code>	All dependencies pinned
<code>.env.example</code>	Environment variable template
<code>pytest.ini</code>	Test configuration
<code>tests/conftest.py</code>	Shared test fixtures (sample docs, tmp chroma dir)
<code>src/__init__.py</code>	Package marker
<code>tests/__init__.py</code>	Package marker
<code>src/chunking.py</code>	<code>split_documents()</code> — wraps RecursiveCharacterTextSplitter
<code>src/ingest.py</code>	<code>load_documents()</code> , <code>build_vector_store()</code> — load PDFs, chunk, embed, persist
<code>src/embedding.py</code>	<code>get_embedding_model()</code> — HuggingFace embeddings factory
<code>src/vector_store.py</code>	<code>get_or_create_store()</code> , <code>get_all_documents()</code> — ChromaDB CRUD
<code>src/retriever.py</code>	<code>dense_retrieve()</code> , <code>sparse_retrieve()</code> , <code>mmr_retrieve()</code>
<code>src/fusion.py</code>	<code>reciprocal_rank_fusion()</code> , <code>apply_mmr()</code>
<code>src/prompt.py</code>	<code>build_prompt()</code> — returns ChatPromptTemplate
<code>src/qa.py</code>	<code>answer_question()</code> — orchestrates retrieval → fusion → generation
<code>src/main.py</code>	CLI entrypoint: <code>python src/main.py --query "..."</code>

File	Responsibility
<code>scripts/download_data.py</code>	Downloads Kaggle dataset to <code>data/documents/</code>
<code>tests/test_chunking.py</code>	Unit tests for chunking
<code>tests/test_ingest.py</code>	Unit tests for document loading
<code>tests/test_embedding.py</code>	Unit tests for embedding model
<code>tests/test_vector_store.py</code>	Unit tests for ChromaDB operations
<code>tests/test_retriever.py</code>	Unit tests for all three retrievers
<code>tests/test_fusion.py</code>	Unit tests for RRF and MMR
<code>tests/test_prompt.py</code>	Unit tests for prompt formatting
<code>tests/test_qa.py</code>	Unit tests for QA chain (mocked LLM)
<code>tests/test_integration.py</code>	End-to-end test (requires API key + documents)

Task 1: Project Setup & Environment

Files:

- Create: `requirements.txt`
- Create: `.env.example`
- Create: `pytest.ini`
- Create: `src/__init__.py`
- Create: `tests/__init__.py`
- Create: `tests/conftest.py`
- ☐ **Step 1: Create requirements.txt**

```
langchain==0.3.25
langchain-community==0.3.24
langchain-chroma==0.2.4
langchain-huggingface==0.1.2
langchain-google-genai==2.1.4
chromadb==0.6.3
sentence-transformers==4.1.0
rank-bm25==0.2.2
pypdf==5.4.0
python-dotenv==1.1.0
numpy==2.2.6
scikit-learn==1.6.1
pytest==8.3.5
```

```
pytest-mock==3.14.0
kaggle==1.7.4
```

- ☐ **Step 2: Create .env.example**

```
GOOGLE_API_KEY=your_google_api_key_here
CHROMA_PERSIST_DIR=./chroma_db
DOCS_DIR=./data/documents
```

Copy to `.env` and fill in your key: `cp .env.example .env`

- ☐ **Step 3: Create pytest.ini**

```
[pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
```

- ☐ **Step 4: Create package markers and directory structure**

```
mkdir data\documents
mkdir chroma_db
type nul > src\__init__.py
type nul > tests\__init__.py
```

- ☐ **Step 5: Create tests/conftest.py**

```
import pytest
from langchain_core.documents import Document

@pytest.fixture
def sample_documents():
    return [
        Document(
            page_content="AI governance refers to the frameworks, policies, and
principles that guide the development and deployment of artificial intelligence
systems.",
            metadata={"source": "governance_framework.pdf", "page": 1},
        ),
        Document(
            page_content="Machine learning models require oversight mechanisms
to ensure they operate within acceptable ethical and legal boundaries.",
```

```

        metadata={"source": "ml_oversight.pdf", "page": 2},
    ),
    Document(
        page_content="Transparency in AI systems is essential for public
trust. Stakeholders must be able to understand how decisions are made by
automated systems.",
        metadata={"source": "transparency_report.pdf", "page": 3},
    ),
    Document(
        page_content="Algorithmic bias can perpetuate existing social
inequalities. Fairness audits are a critical component of responsible AI
deployment.",
        metadata={"source": "governance_framework.pdf", "page": 2},
    ),
    Document(
        page_content="Data privacy regulations such as GDPR and CCPA govern
how AI systems must handle personal information collected from users.",
        metadata={"source": "ml_oversight.pdf", "page": 5},
    ),
    Document(
        page_content="The OECD AI Principles emphasize that AI should be
robust, secure, and safe throughout its entire lifecycle.",
        metadata={"source": "oecd_principles.pdf", "page": 1},
    ),
    Document(
        page_content="Human oversight of AI systems is a key safeguard
against unintended consequences. Humans must remain accountable for AI-assisted
decisions.",
        metadata={"source": "transparency_report.pdf", "page": 7},
    ),
]

@pytest.fixture
def tmp_chroma_dir(tmp_path):
    return str(tmp_path / "chroma_db")

```

- ☐ **Step 6: Install dependencies and verify**

```

pip install -r requirements.txt
python -c "import langchain; import chromadb; import sentence_transformers;
print('OK')"

```

Expected output: OK

- ☐ **Step 7: Commit**

```
git add requirements.txt .env.example pytest.ini src/__init__.py
tests/__init__.py tests/conftest.py
git commit -m "feat: project setup – deps, env template, pytest config,
fixtures"
```

Task 2: Chunking Module

Files:

- Create: `src/chunking.py`
- Create: `tests/test_chunking.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_chunking.py
from langchain_core.documents import Document
from src.chunking import split_documents

def test_split_returns_documents():
    docs = [Document(page_content="word " * 400, metadata={"source": "a.pdf",
"page": 1})]
    chunks = split_documents(docs)
    assert len(chunks) > 1
    assert all(isinstance(c, Document) for c in chunks)

def test_split_respects_chunk_size():
    docs = [Document(page_content="word " * 400, metadata={"source": "a.pdf",
"page": 1})]
    chunks = split_documents(docs, chunk_size=500, chunk_overlap=50)
    for chunk in chunks:
        assert len(chunk.page_content) <= 600 # some tolerance for splitter
boundaries

def test_split_preserves_metadata():
    docs = [Document(page_content="word " * 400, metadata={"source":
"test.pdf", "page": 5})]
    chunks = split_documents(docs)
    assert all(c.metadata["source"] == "test.pdf" for c in chunks)

def test_split_empty_input():
    assert split_documents([]) == []

def test_split_short_document_returns_one_chunk():
```

```
docs = [Document(page_content="Short text.", metadata={"source": "a.pdf",
"page": 1})]
chunks = split_documents(docs, chunk_size=1000, chunk_overlap=200)
assert len(chunks) == 1
assert chunks[0].page_content == "Short text."
```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_chunking.py -v
```

Expected: `ModuleNotFoundError` or `ImportError` — `split_documents` doesn't exist yet.

- ☐ **Step 3: Implement `src/chunking.py`**

```
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

def split_documents(
    documents: list[Document],
    chunk_size: int = 1000,
    chunk_overlap: int = 200,
) -> list[Document]:
    if not documents:
        return []
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        add_start_index=True,
    )
    return splitter.split_documents(documents)
```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_chunking.py -v
```

Expected: 5 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/chunking.py tests/test_chunking.py
git commit -m "feat: chunking module with RecursiveCharacterTextSplitter"
```

Task 3: Document Loader & Ingestion Pipeline

Files:

- Create: `src/ingest.py`
- Create: `tests/test_ingest.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_ingest.py
import os
import pytest
from langchain_core.documents import Document
from src.ingest import load_documents, build_vector_store

def test_load_documents_from_pdf(tmp_path):
    # Create a minimal PDF using reportlab if available, otherwise skip
    pytest.importorskip("pypdf")
    # Use a fixture text file instead to avoid PDF generation dep
    txt_file = tmp_path / "test.txt"
    txt_file.write_text("This is a test document about AI governance frameworks.")
    docs = load_documents(str(tmp_path))
    assert len(docs) >= 1
    assert all(isinstance(d, Document) for d in docs)

def test_load_documents_empty_dir(tmp_path):
    docs = load_documents(str(tmp_path))
    assert docs == []

def test_load_documents_returns_metadata(tmp_path):
    txt_file = tmp_path / "policy.txt"
    txt_file.write_text("AI policy document content here.")
    docs = load_documents(str(tmp_path))
    assert len(docs) >= 1
    assert "source" in docs[0].metadata

def test_build_vector_store_creates_collection(tmp_path, sample_documents):
    from src.embedding import get_embedding_model
    embedding_model = get_embedding_model()
    chroma_dir = str(tmp_path / "chroma")
    store = build_vector_store(sample_documents, chroma_dir, embedding_model)
    assert store is not None
    result = store.similarity_search("AI governance", k=2)
    assert len(result) >= 1
```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_ingest.py -v
```

Expected: `ImportError` — `load_documents` doesn't exist yet.

- ☐ **Step 3: Implement `src/ingest.py`**

```
import os
from langchain_core.documents import Document
from langchain_community.document_loaders import (
    PyPDFDirectoryLoader,
    DirectoryLoader,
    TextLoader,
)
from langchain_chroma import Chroma
from src.chunking import split_documents

def load_documents(docs_dir: str) -> list[Document]:
    """Load all PDF and TXT files from a directory."""
    if not os.path.isdir(docs_dir):
        return []

    documents: list[Document] = []

    pdf_loader = PyPDFDirectoryLoader(docs_dir, silent_errors=True)
    try:
        documents.extend(pdf_loader.load())
    except Exception:
        pass

    txt_loader = DirectoryLoader(
        docs_dir,
        glob="**/*.txt",
        loader_cls=TextLoader,
        loader_kwargs={"encoding": "utf-8"},
        silent_errors=True,
    )
    try:
        documents.extend(txt_loader.load())
    except Exception:
        pass

    return documents

def build_vector_store(
    documents: list[Document],
```



```

    persist_dir: str,
    embedding_model,
    chunk_size: int = 1000,
    chunk_overlap: int = 200,
) -> Chroma:
    """Chunk documents, embed, and persist to ChromaDB."""
    chunks = split_documents(documents, chunk_size=chunk_size,
chunk_overlap=chunk_overlap)
    store = Chroma.from_documents(
        documents=chunks,
        embedding=embedding_model,
        persist_directory=persist_dir,
    )
    return store

```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_ingest.py -v
```

Expected: 4 tests PASSED. (The embedding model will download ~90MB on first run — this is expected.)

- ☐ **Step 5: Commit**

```

git add src/ingest.py tests/test_ingest.py
git commit -m "feat: document loader and ingestion pipeline (PDF + TXT →
ChromaDB)"

```

Task 4: Embedding Module

Files:

- Create: `src/embedding.py`
- Create: `tests/test_embedding.py`
- ☐ **Step 1: Write the failing tests**

```

# tests/test_embedding.py
import pytest
from src.embedding import get_embedding_model

def test_get_embedding_model_returns_model():
    model = get_embedding_model()
    assert model is not None

```

```

def test_embedding_produces_vector():
    model = get_embedding_model()
    embedding = model.embed_query("What is AI governance?")
    assert isinstance(embedding, list)
    assert len(embedding) == 384 # all-MiniLM-L6-v2 dimension
    assert all(isinstance(v, float) for v in embedding)

def test_embedding_consistent():
    model = get_embedding_model()
    emb1 = model.embed_query("test query")
    emb2 = model.embed_query("test query")
    assert emb1 == emb2

def test_embedding_different_texts_differ():
    model = get_embedding_model()
    emb1 = model.embed_query("AI governance frameworks")
    emb2 = model.embed_query("quantum computing hardware")
    assert emb1 != emb2

def test_embed_documents_batch():
    model = get_embedding_model()
    texts = ["doc one", "doc two", "doc three"]
    embeddings = model.embed_documents(texts)
    assert len(embeddings) == 3
    assert all(len(e) == 384 for e in embeddings)

def test_custom_model_name():
    model = get_embedding_model(model_name="BAAI/bge-small-en-v1.5")
    embedding = model.embed_query("test")
    assert len(embedding) == 384

```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_embedding.py -v
```

Expected: `ImportError` — `get_embedding_model` doesn't exist yet.

- ☐ **Step 3: Implement `src/embedding.py`**

```

from functools import lru_cache
from langchain_huggingface import HuggingFaceEmbeddings

```

```
@lru_cache(maxsize=4)
def get_embedding_model(
    model_name: str = "all-MiniLM-L6-v2",
) -> HuggingFaceEmbeddings:
    """Return a cached HuggingFace embedding model."""
    return HuggingFaceEmbeddings(
        model_name=model_name,
        model_kwargs={"device": "cpu"},
        encode_kwargs={"normalize_embeddings": True},
    )
```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_embedding.py -v
```

Expected: 6 tests PASSED. (Model downloads on first run — subsequent runs use cache.)

- ☐ **Step 5: Commit**

```
git add src/embedding.py tests/test_embedding.py
git commit -m "feat: embedding module wrapping HuggingFace all-MiniLM-L6-v2"
```

Task 5: Vector Store (ChromaDB)

Files:

- Create: `src/vector_store.py`
- Create: `tests/test_vector_store.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_vector_store.py
import pytest
from langchain_core.documents import Document
from src.vector_store import get_or_create_store, get_all_documents
from src.embedding import get_embedding_model

@pytest.fixture
def embedding_model():
    return get_embedding_model()
```

```

@pytest.fixture
def populated_store(sample_documents, tmp_chroma_dir, embedding_model):
    from langchain_chroma import Chroma
    return Chroma.from_documents(
        documents=sample_documents,
        embedding=embedding_model,
        persist_directory=tmp_chroma_dir,
    )

def test_get_or_create_store_creates_new(tmp_chroma_dir, sample_documents,
embedding_model):
    store = get_or_create_store(tmp_chroma_dir, embedding_model)
    assert store is not None

def test_get_or_create_store_loads_existing(tmp_chroma_dir, sample_documents,
embedding_model):
    from langchain_chroma import Chroma
    # First: create and populate
    Chroma.from_documents(
        documents=sample_documents,
        embedding=embedding_model,
        persist_directory=tmp_chroma_dir,
    )
    # Second: load without re-adding documents
    store = get_or_create_store(tmp_chroma_dir, embedding_model)
    result = store.similarity_search("AI governance", k=2)
    assert len(result) >= 1

def test_get_all_documents_returns_all(populated_store, sample_documents):
    docs = get_all_documents(populated_store)
    assert len(docs) == len(sample_documents)
    assert all(isinstance(d, Document) for d in docs)

def test_get_all_documents_preserves_content(populated_store,
sample_documents):
    docs = get_all_documents(populated_store)
    contents = {d.page_content for d in docs}
    expected_contents = {d.page_content for d in sample_documents}
    assert contents == expected_contents

def test_get_all_documents_preserves_metadata(populated_store):
    docs = get_all_documents(populated_store)
    assert all("source" in d.metadata for d in docs)

```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_vector_store.py -v
```

Expected: `ImportError` — `get_or_create_store` doesn't exist yet.

- ☐ **Step 3: Implement `src/vector_store.py`**

```
from langchain_chroma import Chroma
from langchain_core.documents import Document

def get_or_create_store(persist_dir: str, embedding_model) -> Chroma:
    """Load existing ChromaDB store or create an empty one."""
    return Chroma(
        persist_directory=persist_dir,
        embedding_function=embedding_model,
    )

def get_all_documents(vector_store: Chroma) -> list[Document]:
    """Fetch every stored chunk – used for BM25 indexing."""
    result = vector_store.get(include=["documents", "metadatas"])
    return [
        Document(page_content=text, metadata=meta or {})
        for text, meta in zip(result["documents"], result["metadatas"])
    ]
```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_vector_store.py -v
```

Expected: 5 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/vector_store.py tests/test_vector_store.py
git commit -m "feat: ChromaDB vector store helpers (create, load, fetch all docs)"
```

Task 6: Retrieval (Dense, Sparse, MMR)

Files:

- Create: `src/retriever.py`

- Create: `tests/test_retriever.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_retriever.py
import pytest
from langchain_core.documents import Document
from langchain_chroma import Chroma
from src.retriever import dense_retrieve, sparse_retrieve, mmr_retrieve
from src.embedding import get_embedding_model

@pytest.fixture
def embedding_model():
    return get_embedding_model()

@pytest.fixture
def populated_store(sample_documents, tmp_chroma_dir, embedding_model):
    return Chroma.from_documents(
        documents=sample_documents,
        embedding=embedding_model,
        persist_directory=tmp_chroma_dir,
    )

def test_dense_retrieve_returns_documents(populated_store):
    results = dense_retrieve(populated_store, "AI governance", k=3)
    assert len(results) == 3
    assert all(isinstance(d, Document) for d in results)

def test_dense_retrieve_relevance(populated_store):
    results = dense_retrieve(populated_store, "transparency in AI systems",
k=3)
    contents = " ".join(d.page_content for d in results).lower()
    assert "transparent" in contents or "ai" in contents

def test_dense_retrieve_k_capped_by_corpus_size(populated_store,
sample_documents):
    results = dense_retrieve(populated_store, "AI", k=100)
    assert len(results) <= len(sample_documents)

def test_sparse_retrieve_returns_documents(sample_documents):
    results = sparse_retrieve(sample_documents, "GDPR privacy regulations",
k=3)
    assert len(results) <= 3
    assert all(isinstance(d, Document) for d in results)
```

```
def test_sparse_retrieve_keyword_match(sample_documents):
    results = sparse_retrieve(sample_documents, "GDPR", k=5)
    assert len(results) >= 1
    assert any("GDPR" in d.page_content for d in results)

def test_sparse_retrieve_empty_corpus():
    results = sparse_retrieve([], "query", k=5)
    assert results == []

def test_mmr_retrieve_returns_documents(populated_store):
    results = mmr_retrieve(populated_store, "AI governance oversight", k=3,
fetch_k=6)
    assert 1 <= len(results) <= 3
    assert all(isinstance(d, Document) for d in results)

def test_mmr_retrieve_diversity(populated_store):
    results = mmr_retrieve(populated_store, "AI governance", k=4, fetch_k=7)
    contents = [d.page_content for d in results]
    # No exact duplicates
    assert len(contents) == len(set(contents))
```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_retriever.py -v
```

Expected: `ImportError` — `dense_retrieve` etc. don't exist yet.

- ☐ **Step 3: Implement `src/retriever.py`**

```
from langchain_chroma import Chroma
from langchain_community.retrievers import BM25Retriever
from langchain_core.documents import Document

def dense_retrieve(vector_store: Chroma, query: str, k: int = 20) ->
list[Document]:
    """Semantic similarity search using dense embeddings."""
    return vector_store.similarity_search(query, k=k)

def sparse_retrieve(
    documents: list[Document],
    query: str,
    k: int = 20,
) -> list[Document]:
```

```

"""Keyword-based BM25 retrieval over a document list."""
if not documents:
    return []
retriever = BM25Retriever.from_documents(documents, k=k)
return retriever.invoke(query)

def mmr_retrieve(
    vector_store: Chroma,
    query: str,
    k: int = 20,
    fetch_k: int = 60,
) -> list[Document]:
    """Maximum Marginal Relevance search for diverse dense candidates."""
    return vector_store.max_marginal_relevance_search(
        query,
        k=k,
        fetch_k=fetch_k,
    )

```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_retriever.py -v
```

Expected: 9 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/retriever.py tests/test_retriever.py
git commit -m "feat: dense, BM25 sparse, and MMR retrievers"
```

Task 7: RRF Fusion

Files:

- Create: `src/fusion.py` (RRF only in this task)
- Create: `tests/test_fusion.py` (RRF tests only)
- ☐ **Step 1: Write the failing tests**

```

# tests/test_fusion.py
import pytest
from langchain_core.documents import Document
from src.fusion import reciprocal_rank_fusion

```



```

@pytest.fixture
def doc_a():
    return Document(page_content="AI governance frameworks are essential.",
metadata={"source": "a.pdf"})

@pytest.fixture
def doc_b():
    return Document(page_content="Machine learning requires oversight.",
metadata={"source": "b.pdf"})

@pytest.fixture
def doc_c():
    return Document(page_content="Transparency builds public trust.", metadata=
{"source": "c.pdf"})

@pytest.fixture
def doc_d():
    return Document(page_content="GDPR governs personal data privacy.",
metadata={"source": "d.pdf"})

def test_rrf_returns_documents(doc_a, doc_b, doc_c):
    result = reciprocal_rank_fusion([[doc_a, doc_b], [doc_b, doc_c]])
    assert len(result) >= 1
    assert all(isinstance(d, Document) for d in result)

def test_rrf_deduplicates(doc_a, doc_b):
    result = reciprocal_rank_fusion([[doc_a, doc_b], [doc_a, doc_b]])
    contents = [d.page_content for d in result]
    assert len(contents) == len(set(contents))

def test_rrf_promotes_consistent_top_rank(doc_a, doc_b, doc_c, doc_d):
    # doc_a is rank 1 in both lists → should score highest
    result = reciprocal_rank_fusion([[doc_a, doc_b, doc_c], [doc_a, doc_d,
doc_b]])
    assert result[0].page_content == doc_a.page_content

def test_rrf_single_list(doc_a, doc_b, doc_c):
    result = reciprocal_rank_fusion([[doc_a, doc_b, doc_c]])
    assert result[0].page_content == doc_a.page_content

def test_rrf_empty_lists():
    result = reciprocal_rank_fusion([], [])
    assert result == []

```

```
def test_rrf_empty_input():
    result = reciprocal_rank_fusion([])
    assert result == []

def test_rrf_preserves_all_unique_docs(doc_a, doc_b, doc_c, doc_d):
    result = reciprocal_rank_fusion([[doc_a, doc_b], [doc_c, doc_d]])
    assert len(result) == 4
```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_fusion.py -v
```

Expected: `ImportError` — `reciprocal_rank_fusion` doesn't exist yet.

- ☐ **Step 3: Implement RRF in src/fusion.py**

```
import hashlib
from langchain_core.documents import Document

def _doc_id(doc: Document) -> str:
    return hashlib.md5(doc.page_content.encode()).hexdigest()

def reciprocal_rank_fusion(
    result_lists: list[list[Document]],
    k: int = 60,
) -> list[Document]:
    """
    Fuse multiple ranked result lists using Reciprocal Rank Fusion.
    k=60 is the standard constant that dampens rank influence (from the
    original RRF paper).
    Higher k → more uniform weighting across ranks.
    """
    if not result_lists:
        return []

    scores: dict[str, float] = {}
    doc_map: dict[str, Document] = {}

    for result_list in result_lists:
        for rank, doc in enumerate(result_list, start=1):
            doc_id = _doc_id(doc)
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
            doc_map[doc_id] = doc
```

```
sorted_ids = sorted(scores, key=lambda x: scores[x], reverse=True)
return [doc_map[doc_id] for doc_id in sorted_ids]
```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_fusion.py -v
```

Expected: 7 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/fusion.py tests/test_fusion.py
git commit -m "feat: Reciprocal Rank Fusion (k=60) for hybrid retrieval"
```

Task 8: MMR Diversification

Files:

- Modify: `src/fusion.py` (add `apply_mmr`)
- Modify: `tests/test_fusion.py` (add MMR tests)
- ☐ **Step 1: Write the failing tests (append to `tests/test_fusion.py`)**

```
# Append these to the bottom of tests/test_fusion.py
from src.fusion import apply_mmr
from src.embedding import get_embedding_model

@pytest.fixture
def embedding_model():
    return get_embedding_model()

@pytest.fixture
def diverse_documents():
    return [
        Document(page_content="AI governance frameworks establish
accountability.", metadata={"source": "a.pdf"}),
        Document(page_content="AI governance policies define oversight roles.",
metadata={"source": "b.pdf"}), # similar to first
        Document(page_content="GDPR requires data minimization and user
consent.", metadata={"source": "c.pdf"}), # different topic
        Document(page_content="Machine learning bias can harm marginalized
```

```

groups.", metadata={"source": "d.pdf"}),
    Document(page_content="AI governance accountability is central to
ethics.", metadata={"source": "e.pdf"}), # similar to first
    Document(page_content="Quantum computing will reshape cryptography.",
metadata={"source": "f.pdf"}), # very different
]

def test_apply_mmr_returns_k_documents(diverse_documents, embedding_model):
    result = apply_mmr("AI governance", diverse_documents, embedding_model,
k=3)
    assert len(result) == 3
    assert all(isinstance(d, Document) for d in result)

def test_apply_mmr_no_duplicates(diverse_documents, embedding_model):
    result = apply_mmr("AI governance", diverse_documents, embedding_model,
k=4)
    contents = [d.page_content for d in result]
    assert len(contents) == len(set(contents))

def test_apply_mmr_k_larger_than_candidates(diverse_documents,
embedding_model):
    result = apply_mmr("AI governance", diverse_documents[:2], embedding_model,
k=10)
    assert len(result) == 2 # capped at available candidates

def test_apply_mmr_empty_candidates(embedding_model):
    result = apply_mmr("query", [], embedding_model, k=5)
    assert result == []

def test_apply_mmr_selects_diverse_results(diverse_documents, embedding_model):
    # With lambda_mult=0.5, MMR should NOT select two near-identical docs
    result = apply_mmr("AI governance accountability", diverse_documents,
embedding_model, k=3, lambda_mult=0.5)
    # Docs 0 and 4 are semantically near-identical ("AI governance ...
accountability")
    selected_contents = [d.page_content for d in result]
    near_identical_count = sum(
        1 for c in selected_contents
        if "AI governance" in c and "accountab" in c.lower()
    )
    # MMR should select at most 1 of the near-identical docs
    assert near_identical_count <= 1

```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_fusion.py -v -k "mmr"
```

Expected: `ImportError` — `apply_mmr` doesn't exist in `src/fusion.py` yet.

- ☐ **Step 3: Implement `apply_mmr` in `src/fusion.py` (append to existing file)**

```
# Add these imports at the top of src/fusion.py (after existing imports):
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

def apply_mmr(
    query: str,
    candidates: list[Document],
    embedding_model,
    k: int = 8,
    lambda_mult: float = 0.5,
) -> list[Document]:
    """
    Maximum Marginal Relevance diversification over a candidate list.
    lambda_mult: 1.0 = pure relevance, 0.0 = pure diversity.
    """
    if not candidates:
        return []

    k = min(k, len(candidates))

    query_emb = np.array(embedding_model.embed_query(query)).reshape(1, -1)
    doc_embs = np.array(embedding_model.embed_documents([d.page_content for d
in candidates]))

    relevance_scores = cosine_similarity(query_emb, doc_embs)[0]

    selected_indices: list[int] = []
    remaining = list(range(len(candidates)))

    while len(selected_indices) < k and remaining:
        if not selected_indices:
            best = max(remaining, key=lambda i: relevance_scores[i])
        else:
            selected_embs = doc_embs[selected_indices]
            mmr_scores: dict[int, float] = {}
            for i in remaining:
                rel = relevance_scores[i]
                redundancy = float(
                    cosine_similarity(doc_embs[i].reshape(1, -1),
selected_embs).max()
                )
                mmr_scores[i] = lambda_mult * rel - (1 - lambda_mult) *
redundancy
            best = max(remaining, key=lambda i: mmr_scores[i])
```

```
selected_indices.append(best)
remaining.remove(best)

return [candidates[i] for i in selected_indices]
```

- ☐ **Step 4: Run all fusion tests to verify they pass**

```
pytest tests/test_fusion.py -v
```

Expected: 12 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/fusion.py tests/test_fusion.py
git commit -m "feat: MMR diversification post-RRF for diverse context
selection"
```

Task 9: Prompt Template

Files:

- Create: `src/prompt.py`
- Create: `tests/test_prompt.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_prompt.py
from langchain_core.documents import Document
from src.prompt import build_prompt, format_context

def test_build_prompt_returns_template():
    from langchain_core.prompts import ChatPromptTemplate
    prompt = build_prompt()
    assert isinstance(prompt, ChatPromptTemplate)

def test_prompt_has_context_variable():
    prompt = build_prompt()
    assert "context" in prompt.input_variables

def test_prompt_has_question_variable():
    prompt = build_prompt()
```

```

assert "question" in prompt.input_variables

def test_format_context_includes_source():
    docs = [
        Document(page_content="AI governance is important.", metadata=
{"source": "policy.pdf", "page": 3}),
    ]
    context = format_context(docs)
    assert "AI governance is important." in context
    assert "policy.pdf" in context

def test_format_context_multiple_docs():
    docs = [
        Document(page_content="First doc content.", metadata={"source":
"a.pdf", "page": 1}),
        Document(page_content="Second doc content.", metadata={"source":
"b.pdf", "page": 2}),
    ]
    context = format_context(docs)
    assert "First doc content." in context
    assert "Second doc content." in context
    assert "a.pdf" in context
    assert "b.pdf" in context

def test_format_context_missing_metadata():
    docs = [Document(page_content="Content only.", metadata={})]
    context = format_context(docs)
    assert "Content only." in context

def test_prompt_renders_with_values():
    prompt = build_prompt()
    messages = prompt.format_messages(context="Some context.", question="What
is AI?")
    full_text = " ".join(m.content for m in messages)
    assert "Some context." in full_text
    assert "What is AI?" in full_text

```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_prompt.py -v
```

Expected: **ImportError** — `build_prompt` doesn't exist yet.

- ☐ **Step 3: Implement `src/prompt.py`**

```

from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate

_SYSTEM_TEMPLATE = """You are an AI assistant answering questions using only
the provided context from AI governance documents.

Instructions:
1. Use ONLY the retrieved context below. Do not use outside knowledge.
2. If the answer is not found in the context, respond with exactly: "I cannot
find sufficient evidence in the provided documents to answer this question."
3. Cite the source document and page for each claim you make.
4. Be concise and factual.

Context:
{context}"""

_HUMAN_TEMPLATE = "Question: {question}"

def build_prompt() -> ChatPromptTemplate:
    return ChatPromptTemplate.from_messages(
        [
            ("system", _SYSTEM_TEMPLATE),
            ("human", _HUMAN_TEMPLATE),
        ]
    )

def format_context(documents: list[Document]) -> str:
    """Format retrieved documents into a numbered context block with source
    citations."""
    parts = []
    for i, doc in enumerate(documents, start=1):
        source = doc.metadata.get("source", "Unknown")
        page = doc.metadata.get("page", "")
        citation = f"[{i}] Source: {source}" + (f", Page {page}" if page else "")
        parts.append(f"{citation}\n{doc.page_content}")
    return "\n\n".join(parts)

```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_prompt.py -v
```

Expected: 7 tests PASSED.

- ☐ **Step 5: Commit**


```
git add src/prompt.py tests/test_prompt.py
git commit -m "feat: prompt template with source citation formatting"
```

Task 10: QA Chain & Answer Generation

Files:

- Create: `src/qa.py`
- Create: `tests/test_qa.py`
- ☐ **Step 1: Write the failing tests**

```
# tests/test_qa.py
import pytest
from unittest.mock import MagicMock, patch
from langchain_core.documents import Document
from langchain_core.messages import AIMessage
from src.qa import answer_question, format_sources

@pytest.fixture
def mock_llm():
    llm = MagicMock()
    llm.invoke.return_value = AIMessage(content="AI governance refers to
frameworks and policies.")
    return llm

@pytest.fixture
def retrieved_docs():
    return [
        Document(
            page_content="AI governance refers to the frameworks and policies
that guide AI development.",
            metadata={"source": "governance.pdf", "page": 1},
        ),
        Document(
            page_content="Transparency requirements are central to AI
governance frameworks.",
            metadata={"source": "transparency.pdf", "page": 5},
        ),
    ]

def test_answer_question_returns_dict(mock_llm, retrieved_docs):
    result = answer_question(
        query="What is AI governance?",
        retrieved_docs=retrieved_docs,
```

```

        llm=mock_llm,
    )
    assert isinstance(result, dict)

def test_answer_question_has_answer_key(mock_llm, retrieved_docs):
    result = answer_question("What is AI governance?", retrieved_docs,
mock_llm)
    assert "answer" in result
    assert isinstance(result["answer"], str)

def test_answer_question_has_sources_key(mock_llm, retrieved_docs):
    result = answer_question("What is AI governance?", retrieved_docs,
mock_llm)
    assert "sources" in result
    assert isinstance(result["sources"], list)

def test_answer_question_has_query_key(mock_llm, retrieved_docs):
    result = answer_question("What is AI governance?", retrieved_docs,
mock_llm)
    assert result["query"] == "What is AI governance?"

def test_answer_question_sources_contain_metadata(mock_llm, retrieved_docs):
    result = answer_question("What is AI governance?", retrieved_docs,
mock_llm)
    for source in result["sources"]:
        assert "source" in source
        assert "content" in source

def test_answer_question_sources_count_matches_docs(mock_llm, retrieved_docs):
    result = answer_question("What is AI governance?", retrieved_docs,
mock_llm)
    assert len(result["sources"]) == len(retrieved_docs)

def test_format_sources():
    docs = [
        Document(page_content="Content.", metadata={"source": "a.pdf", "page":
3}),
        Document(page_content="More content.", metadata={"source": "b.pdf",
"page": 7}),
    ]
    sources = format_sources(docs)
    assert len(sources) == 2
    assert sources[0]["source"] == "a.pdf"
    assert sources[0]["page"] == 3
    assert sources[0]["content"] == "Content."
    assert sources[1]["source"] == "b.pdf"

```

```
def test_answer_question_calls_llm(mock_llm, retrieved_docs):
    answer_question("test query", retrieved_docs, mock_llm)
    assert mock_llm.invoke.called
```

- ☐ **Step 2: Run tests to verify they fail**

```
pytest tests/test_qa.py -v
```

Expected: `ImportError` — `answer_question` doesn't exist yet.

- ☐ **Step 3: Implement `src/qa.py`**

```
from langchain_core.documents import Document
from langchain_core.messages import BaseMessage
from src.prompt import build_prompt, format_context

def format_sources(documents: list[Document]) -> list[dict]:
    return [
        {
            "source": doc.metadata.get("source", "Unknown"),
            "page": doc.metadata.get("page", None),
            "content": doc.page_content,
        }
        for doc in documents
    ]

def answer_question(
    query: str,
    retrieved_docs: list[Document],
    llm,
) -> dict:
    """Generate a grounded answer from retrieved documents using the LLM."""
    prompt = build_prompt()
    context = format_context(retrieved_docs)
    messages = prompt.format_messages(context=context, question=query)
    response: BaseMessage = llm.invoke(messages)
    return {
        "query": query,
        "answer": response.content,
        "sources": format_sources(retrieved_docs),
    }
```

- ☐ **Step 4: Run tests to verify they pass**

```
pytest tests/test_qa.py -v
```

Expected: 9 tests PASSED.

- ☐ **Step 5: Commit**

```
git add src/qa.py tests/test_qa.py
git commit -m "feat: QA chain – grounded answer generation with source attribution"
```

Task 11: Main CLI

Files:

- Create: `src/main.py`

This is the orchestration layer. No new tests (covered by integration test in Task 12).

- ☐ **Step 1: Implement `src/main.py`**

```
"""
Usage:
    python src/main.py --query "What is AI governance?"
    python src/main.py --query "What does OECD say about AI?" --top-k 5
    python src/main.py --ingest # Re-index documents from data/documents/
"""

import argparse
import os
from dotenv import load_dotenv

load_dotenv()

from src.embedding import get_embedding_model
from src.ingest import load_documents, build_vector_store
from src.vector_store import get_or_create_store, get_all_documents
from src.retriever import dense_retrieve, sparse_retrieve, mmr_retrieve
from src.fusion import reciprocal_rank_fusion, apply_mmr
from src.qa import answer_question

def _get_llm():
    from langchain_google_genai import ChatGoogleGenerativeAI
    api_key = os.getenv("GOOGLE_API_KEY")
    if not api_key:
        raise ValueError("GOOGLE_API_KEY not set. Copy .env.example to .env and add your key.")
    return ChatGoogleGenerativeAI(
```

```

        model="gemini-2.5-flash",
        temperature=0,
        google_api_key=api_key,
    )

def ingest(docs_dir: str, chroma_dir: str) -> None:
    print(f"Loading documents from {docs_dir}...")
    documents = load_documents(docs_dir)
    if not documents:
        print(f"No documents found in {docs_dir}. Run scripts/download_data.py first.")
        return
    print(f"Loaded {len(documents)} pages. Chunking and indexing...")
    embedding_model = get_embedding_model()
    store = build_vector_store(documents, chroma_dir, embedding_model)
    all_docs = get_all_documents(store)
    print(f"Indexed {len(all_docs)} chunks into ChromaDB at {chroma_dir}")

def query(question: str, chroma_dir: str, top_k: int = 8) -> None:
    embedding_model = get_embedding_model()
    store = get_or_create_store(chroma_dir, embedding_model)
    all_docs = get_all_documents(store)

    if not all_docs:
        print("ChromaDB is empty. Run: python src/main.py --ingest")
        return

    print(f"Retrieving context for: {question!r}")

    dense_results = dense_retrieve(store, question, k=20)
    sparse_results = sparse_retrieve(all_docs, question, k=20)
    mmr_results = mmr_retrieve(store, question, k=20, fetch_k=60)

    fused = reciprocal_rank_fusion([dense_results, sparse_results,
mmr_results])
    final_docs = apply_mmr(question, fused, embedding_model, k=top_k)

    print(f"Using {len(final_docs)} context chunks.\n")

    llm = _get_llm()
    result = answer_question(question, final_docs, llm)

    print("=" * 60)
    print(f"Answer:\n{result['answer']}")
    print("\nSources:")
    for src in result["sources"]:
        page_info = f", Page {src['page']}" if src.get("page") else ""
        print(f" - {src['source']}{page_info}")
    print("=" * 60)

```

```
def main() -> None:
    parser = argparse.ArgumentParser(description="RAG system for AI governance documents")
    parser.add_argument("--query", "-q", type=str, help="Question to answer")
    parser.add_argument("--ingest", action="store_true", help="Re-ingest documents")
    parser.add_argument("--top-k", type=int, default=8, help="Number of context chunks (default: 8)")
    args = parser.parse_args()

    chroma_dir = os.getenv("CHROMA_PERSIST_DIR", "./chroma_db")
    docs_dir = os.getenv("DOCS_DIR", "./data/documents")

    if args.ingest:
        ingest(docs_dir, chroma_dir)
    elif args.query:
        query(args.query, chroma_dir, top_k=args.top_k)
    else:
        parser.print_help()

if __name__ == "__main__":
    main()
```

- ☐ **Step 2: Verify the CLI help works**

```
python src/main.py --help
```

Expected output:

```
usage: main.py [-h] [--query QUERY] [--ingest] [--top-k TOP_K]

RAG system for AI governance documents

options:
  -h, --help            show this help message and exit
  --query QUERY, -q QUERY Question to answer
  --ingest              Re-ingest documents
  --top-k TOP_K         Number of context chunks (default: 8)
```

- ☐ **Step 3: Commit**

```
git add src/main.py
git commit -m "feat: CLI entrypoint – ingest and query commands"
```

Task 12: Dataset Download & Integration Test

Files:

- Create: `scripts/download_data.py`
- Create: `tests/test_integration.py`
- ☐ **Step 1: Create `scripts/download_data.py`**

```
"""
Download the AI governance documents dataset from Kaggle.

Prerequisites:
1. Create a Kaggle account at kaggle.com
2. Go to Account → API → Create New Token → download kaggle.json
3. Place kaggle.json at ~/.kaggle/kaggle.json (Linux/Mac)
   or C:\\Users\\<user>\\.kaggle\\kaggle.json (Windows)
4. pip install kaggle (already in requirements.txt)

Usage:
    python scripts/download_data.py
"""

import os
import zipfile
from pathlib import Path

DATASET = "umerhaddii/ai-governance-documents-data"
OUTPUT_DIR = Path("data/documents")

def download():
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    try:
        import kaggle
        kaggle.api.authenticate()
        print(f"Downloading {DATASET}...")
        kaggle.api.dataset_download_files(DATASET, path=str(OUTPUT_DIR),
unzip=True)
        print(f"Dataset downloaded to {OUTPUT_DIR}")
    except Exception as e:
        print(f"Kaggle download failed: {e}")
        print("\nAlternative: use kagglehub:")
        print("    pip install kagglehub")
        print("    python -c \"import kagglehub;")
print(kagglehub.dataset_download('umerhaddii/ai-governance-documents-
data'))\"")
        print("Then copy the files to data/documents/")
        raise
```

```
if __name__ == "__main__":  
    download()
```

- ☐ **Step 2: Run dataset download**

```
python scripts/download_data.py
```

Expected: Files appear in `data/documents/`. If Kaggle credentials aren't configured, follow the printed instructions.

After download, verify:

```
ls data/documents/
```

Expected: Several PDF files related to AI governance.

- ☐ **Step 3: Ingest the documents**

```
python src/main.py --ingest
```

Expected output:

```
Loading documents from ./data/documents...  
Loaded N pages. Chunking and indexing...  
Indexed M chunks into ChromaDB at ./chroma_db
```

- ☐ **Step 4: Write the integration test**

```
# tests/test_integration.py  
"""  
Integration tests – require:  
1. Documents in data/documents/  
2. GOOGLE_API_KEY in .env  
  
Skip with: pytest tests/test_integration.py -m "not integration"  
Run with:  pytest tests/test_integration.py -v -m integration  
"""  
  
import os  
import pytest  
from dotenv import load_dotenv  
  
load_dotenv()
```



```

@pytest.mark.integration
def test_full_pipeline_retrieval(tmp_path):
    """Test retrieval pipeline without LLM call."""
    from src.embedding import get_embedding_model
    from src.ingest import load_documents, build_vector_store
    from src.vector_store import get_all_documents
    from src.retriever import dense_retrieve, sparse_retrieve, mmr_retrieve
    from src.fusion import reciprocal_rank_fusion, apply_mmr

    docs_dir = "data/documents"
    if not os.path.isdir(docs_dir) or not os.listdir(docs_dir):
        pytest.skip("data/documents is empty – run scripts/download_data.py first")

    embedding_model = get_embedding_model()
    chroma_dir = str(tmp_path / "chroma")

    documents = load_documents(docs_dir)
    assert len(documents) > 0, "No documents loaded"

    store = build_vector_store(documents, chroma_dir, embedding_model)
    all_docs = get_all_documents(store)
    assert len(all_docs) > 0

    query = "What are the key principles of AI governance?"

    dense_results = dense_retrieve(store, query, k=10)
    sparse_results = sparse_retrieve(all_docs, query, k=10)
    mmr_results = mmr_retrieve(store, query, k=10, fetch_k=30)

    assert len(dense_results) > 0
    assert len(sparse_results) > 0
    assert len(mmr_results) > 0

    fused = reciprocal_rank_fusion([dense_results, sparse_results,
mmr_results])
    assert len(fused) > 0

    final = apply_mmr(query, fused, embedding_model, k=5)
    assert 1 <= len(final) <= 5

    # No duplicate content in final results
    contents = [d.page_content for d in final]
    assert len(contents) == len(set(contents))

@pytest.mark.integration
def test_full_pipeline_with_llm(tmp_path):
    """End-to-end test including LLM call. Requires GOOGLE_API_KEY."""
    api_key = os.getenv("GOOGLE_API_KEY")
    if not api_key:

```

```

pytest.skip("GOOGLE_API_KEY not set")

docs_dir = "data/documents"
if not os.path.isdir(docs_dir) or not os.listdir(docs_dir):
    pytest.skip("data/documents is empty – run scripts/download_data.py
first")

from langchain_google_genai import ChatGoogleGenerativeAI
from src.embedding import get_embedding_model
from src.ingest import load_documents, build_vector_store
from src.vector_store import get_all_documents
from src.retriever import dense_retrieve, sparse_retrieve, mmr_retrieve
from src.fusion import reciprocal_rank_fusion, apply_mmr
from src.qa import answer_question

embedding_model = get_embedding_model()
chroma_dir = str(tmp_path / "chroma")
documents = load_documents(docs_dir)
store = build_vector_store(documents, chroma_dir, embedding_model)
all_docs = get_all_documents(store)

query = "What is AI governance and why does it matter?"

dense_results = dense_retrieve(store, query, k=20)
sparse_results = sparse_retrieve(all_docs, query, k=20)
mmr_results = mmr_retrieve(store, query, k=20, fetch_k=60)
fused = reciprocal_rank_fusion([dense_results, sparse_results,
mmr_results])
final = apply_mmr(query, fused, embedding_model, k=8)

llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0,
google_api_key=api_key)
result = answer_question(query, final, llm)

assert "answer" in result
assert len(result["answer"]) > 50 # non-trivial answer
assert "sources" in result
assert len(result["sources"]) > 0
# Answer should not be the fallback (we have documents)
assert "cannot find sufficient evidence" not in result["answer"].lower()

```

- ☐ **Step 5: Run unit tests (all should pass without API key)**

```
pytest tests/ -v --ignore=tests/test_integration.py
```

Expected: All tests PASSED.

- ☐ **Step 6: Run integration retrieval test (no API key needed)**

```
pytest tests/test_integration.py::test_full_pipeline_retrieval -v -m
integration
```

Expected: PASSED (requires documents in [data/documents/](#)).

- ☐ **Step 7: Run full end-to-end test (requires API key)**

```
pytest tests/test_integration.py::test_full_pipeline_with_llm -v -m integration
```

Expected: PASSED.

- ☐ **Step 8: Run a live query**

```
python src/main.py --query "What are the main principles of the OECD AI
governance framework?"
```

Expected: Printed answer with cited sources.

- ☐ **Step 9: Commit**

```
git add scripts/download_data.py tests/test_integration.py
git commit -m "feat: dataset download script and end-to-end integration tests"
```

Self-Review

Spec Coverage Check:

Requirement	Covered by
Document ingestion	Task 3 (ingest.py)
Parsing and chunking	Task 2 (chunking.py)
Generate embeddings	Task 4 (embedding.py)
Store searchable representations	Task 5 (vector_store.py)
Dense retrieval (top-20)	Task 6 (retriever.py)
Sparse retrieval (top-20)	Task 6 (retriever.py , BM25)
RRF fusion	Task 7 (fusion.py)
MMR diversification (k=8)	Task 8 (fusion.py)

Requirement	Covered by
Grounded answer generation	Task 10 (<code>qa.py</code>)
Source attribution	Task 10 (<code>qa.py</code> , <code>format_sources</code>)
Temperature=0	Task 11 (<code>main.py</code> , <code>_get_llm</code>)
Fallback "cannot find evidence"	Task 9 (<code>prompt.py</code> , system template)
ChromaDB persistence	Task 5 (<code>vector_store.py</code> , <code>persist_directory</code>)
CLI interface	Task 11 (<code>main.py</code>)
Kaggle dataset download	Task 12 (<code>scripts/download_data.py</code>)
Multi-vector retrieval	Covered via three retrievers (dense + sparse + MMR) fused with RRF

Placeholder Scan: None found — all steps have complete code.

Type Consistency Check:

- `split_documents()` → `list[Document]` ✓ used in `ingest.py`
- `get_embedding_model()` → `HuggingFaceEmbeddings` ✓ used in all retriever/fusion calls
- `dense_retrieve()` → `list[Document]` ✓ fed into `reciprocal_rank_fusion()`
- `reciprocal_rank_fusion()` takes `list[list[Document]]` ✓ matches all call sites in `main.py`
- `apply_mmr()` takes `(str, list[Document], embedding_model, k, lambda_mult)` ✓ matches `main.py`
- `answer_question()` takes `(str, list[Document], llm)` → `dict` ✓ matches `main.py`
- `format_sources()` → `list[dict]` ✓ used in `answer_question()` return value